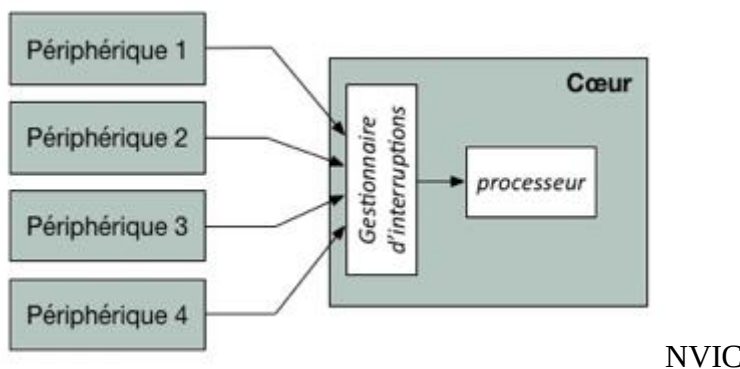


Gérez vos interruptions

Nous allons voir maintenant comment utiliser une interruption avec les périphériques. Rappelons qu'une interruption consiste à suspendre le flot d'exécution du processeur pour réaliser un nouveau traitement avant de reprendre là où il avait été suspendu.

Le chemin d'une interruption

Classiquement une interruption est provoquée par une source extérieure au processeur comme un périphérique. Le processeur doit donc être capable de différencier les différentes sources pour réaliser le traitement voulu. Pour gérer cela, le processeur dispose d'un **gestionnaire d'interruption** (le NVIC) dont le rôle est d'associer à chaque interruption sa fonction de traitement.



Comme nous l'avons vu, pour utiliser le mécanisme d'interruption en provenance d'un périphérique, il va falloir :

- autoriser le périphérique à lancer une interruption ;
- configurer le processeur pour gérer l'interruption ;
- écrire le code associé à l'interruption.

Pour illustrer ces trois étapes, nous allons réaliser le clignotement de la LED vu dans le chapitre précédent, non plus par scrutation, mais avec une interruption. Pour cela, commençons avec le code suivant pour lequel le timer 2 est configuré pour déborder toutes les secondes et la broche 5 du port A est configurée en sortie pour contrôler la LED USER.

```
#include "stm32f10x.h"

int main (void)
{
    // Configuration de la broche en sortie

    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    GPIOA->CRL &= ~(0xF << 4*5);

    GPIOA->CRL |= (0x1 << 4*5);

    // Configuration du timer

    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

    TIM2->ARR = 9999;

    TIM2->PSC = 7199;

    // Lancement du timer

    TIM2->CR1 |= TIM1_CR1_CEN;

    while(1)
    {

    }

    return 0;
}
```

Autorisez l'interruption sur un périphérique

Le débordement d'un timer peut être la source d'une interruption. Ainsi, la documentation (p. 409) nous indique que le bit 0, noté UIE, du registre DIER, permet de contrôler le lancement de l'interruption d'un timer. Il faut donc le passer à 1.

Ainsi en ajoutant, la ligne suivante avant le lancement du timer :

```
...  
  
TIM2->DIER = TIM2->DIER | (1 << 0);  
  
TIM2->CR1 |= TIM1_CR1_CEN;  
  
...
```

ou de manière équivalente

```
TIM2->DIER |= TIM_DIER_UIE;
```

une demande d'interruption sera ainsi transmise au NVIC à chaque fois que le timer déborde.

Configurez l'interruption dans le cœur

Chaque source d'interruption est identifiée par un numéro. Ce numéro est nécessaire pour configurer la gestion des interruptions au niveau du processeur. Ainsi, le tableau p. 197 à 199 indique pour chaque périphérique le numéro de l'interruption qui lui est associé. L'interruption du timer 2 est ainsi identifiée par le numéro 28.

Les informations pour configurer les interruptions au niveau du cœur ne sont pas disponibles dans le document RM0008, mais dans le Programming manuel (PM0056) qui décrit les informations liées au Cortex M3. Toutes les informations nécessaires à la configuration des interruptions sont décrites dans le chapitre dédié au Nested vectored interrupt controller (NVIC), page 118. Le NVIC comprend un ensemble de registres au même titre que ceux des périphériques, à la seule différence qu'ils appartiennent au cœur.

La déclaration des emplacements mémoires de ces registres est spécifiée dans le fichier core_cm3.h qui est inclus dans le fichier stm32f10x.h.

La documentation sur le Cortex M3 nous confirme que le groupe de registres ISER a pour fonction d'activer et de désactiver l'autorisation du traitement des différentes interruptions.

Chaque bit de ces registres est associé à une interruption. Comme le nombre d'interruptions sur le Cortex M3 est de 81, ce groupe de registres est décomposé en trois éléments décrits dans la structure NVIC par un tableau. Ainsi pour activer l'interruption 28, il faut mettre à 1 le bit 28 du registre ISER[0], soit

```
NVIC->ISER[0] = NVIC->ISER[0] | (1 << 28);
```

OU

```
NVIC->ISER[0] = NVIC->ISER[0] | NVIC_ISER_28;
```

Cela suffit pour activer une interruption au niveau du cœur. Rappelez-vous cependant que chaque interruption est associée à une priorité. Cette priorité sert à déterminer l'ordre dans lequel les interruptions sont traitées si plusieurs sont actives en même temps.

Le niveau de priorité est attribué aux interruptions à l'aide du groupe de registres IPR du NVIC. De la même manière que ISER ces registres sont structurés sous forme de tableau. Il existe même dans le fichier *core_cm3.h* une union nommée IP permettant d'accéder directement à l'emplacement des bits de configuration des priorités. Par exemple `NVIC->IP[28]` permet de fixer la priorité de l'interruption 28.

Vous avez peut-être aussi retenu que les priorités sont codées sur 8 bits, mais que seules les bits 7 à 4 sont pris en considération... Donc si on veut, par exemple, fixer la priorité à 7 de l'interruption 28 il faut ajouter le code :

```
NVIC->IP[28] = NVIC->IP[28] | (7 << 4);
```

Associez du code à l'interruption

Nous avons maintenant le code suivant

```
#include <stm32f10x.h>

int main (void)
{
    // Configuration de la broche en sortie

    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    GPIOA->CRL &= ~(0xF << 4*5);

    GPIOA->CRL |= (0x1 << 4*5);
```

```

// Configuration du timer

RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

TIM2->ARR = 9999;

TIM2->PSC = 7199;

TIM2->DIER |= TIM_DIER_UIE;


//Configuration de l'interruption sur le coeur

NVIC->ISER[0] |= NVIC_ISER_SETENA_28;

NVIC->IP[28] |= (7 << 4);


// Lancement du timer

TIM2->CR1 |= TIM_CR1_CEN;

while(1)

{

}

return 0;
}

```

Ce programme permet au timer 2 de lancer une interruption et au cœur de la traiter. Il faut à cela ajouter le code qui sera réalisé pendant cette interruption.

La fonction d'interruption du timer 2 est prototypée par

```
void TIM2_IRQHandler(void);
```

Le nom TIM2_IRQHandler est l'étiquette associée à l'adresse mémoire de l'interruption (déclarée dans le fichier *stm32f10x.s*). Lors de la compilation, cette étiquette prend la valeur de l'adresse mémoire de la fonction TIM2_IRQHandler. Ainsi, quand l'interruption survient, le processeur lit

l'adresse contenue dans TIM2_IRQHandler et saute à cette adresse, permettant ainsi l'exécution de la fonction.

Revenons à la déclaration de la fonction d'interruption et ajoutons le code nécessaire au traitement que l'on souhaite réaliser. Le but est de faire clignoter la LED branchée sur la broche 5 du port A, soit :

```
void TIM2_IRQHandler(void) {  
  
    GPIOA->ODR = GPIOA->ODR ^ (1 << 5);  
  
}
```

Ce code n'est cependant pas opérationnel car l'interruption est toujours valide au niveau du périphérique et donc une nouvelle interruption va immédiatement être relancée quand le traitement du *handler* sera terminé, bouclant ainsi indéfiniment dans la fonction d'interruption.

Il faut donc valider le traitement dans la fonction d'interruption en mettant à 0 le bit UIF du timer 2, soit :

```
void TIM2_IRQHandler(void) {  
  
    TIM2->SR &= ~TIM_SR_UIF;  
  
    GPIOA->ODR = GPIOA->ODR ^ (1 << 5);  
  
}
```

Vous pouvez maintenant compiler et exécuter le code en debug pour observer le clignotement de la LED, ainsi que l'activation de l'interruption.

Scruter ou interrompre un événement

Mais pourquoi s'embêter avec une interruption alors que mon code fait exactement la même chose qu'avant en scrutant l'état du timer ?

C'est vrai, mais la scrutation est globalement inefficace car le processeur va passer la plupart de son temps à attendre qu'un état change, alors qu'il pourrait faire autre chose !

De plus, l'interruption permet un traitement plus réactif car le traitement va survenir dès que l'interruption se produit. Alors que dans le cas de la scrutation, il faut attendre que toute la boucle soit parcourue avant de détecter le changement (dans notre cas la boucle ne fait pas grand chose, mais le traitement pourrait être beaucoup plus long). Si le signal à observer change très vite, il se pourrait donc que la scrutation ne permette pas d'observer ce changement.